Machine Learning B (2025)
Home Assignment 5

Yasin Baysal, cmv882

Contents

1 PAC-Bayesian Aggregation

In this exercise, we will reproduce an experiment from Thiemann et al. (2017) and replicate Figure 2 from their Section 6 on the `Ionosphere` dataset. We will train an ensemble of weak RBF-kernel SVMs on random subsets of the training data and then determine their voting weights by minimizing the PAC-Bayes–λ bound. We start by introducing the data:

Data Preparation

Since we are asked to only reproduce the experiment for the first dataset, `Ionosphere`, we start by downloading the dataset from the UCI repository[1]. The data consist of 351 radar returns, each described by $d = 34$ continuous real-valued features (autocorrelation magnitudes at different pulse-pair delays) and a binary label "g" (good) or "b" (bad). Here, "g" radar returns are those showing evidence of some type of structure in the ionosphere, while "b" returns are those that do not, i.e., their signals pass through the ionosphere.

After downloading the data, we recode the labels "good" as $+1$ and "bad" as -1, such that the labels $y \in \{-1, +1\}$. Hereafter, we standardize each of $d = 34$ features to zero mean and unit variance on the training set, then randomly split the dataset into a training set S of size $|S| = n = 200$ and a test set T of size $|T| = 150$, as in Table 1 of the paper.

```python
def load_ionosphere(path="ionosphere.data", seed=42):
    cols = [f"f{i+1}" for i in range(34)] + ["label"]
    df = pd.read_csv(path, header=None, names=cols)
    df["label"] = df["label"].map({"g": +1, "b": -1})
    X = df.iloc[:, :-1].values
    y = df["label"].values
    X = StandardScaler().fit_transform(X)
    X_tr, X_te, y_tr, y_te = train_test_split(
        X, y, train_size=200, test_size=150, stratify=y, random_state=seed
    )
    return X_tr, X_te, y_tr, y_te
```

Listing 1: Data preparation for the `Ionosphere` dataset.

The PAC-Bayes-λ Bound

In the experiment, we focus on PAC-Bayesian aggregation that refers to prediction with ρ-weighted majority vote. Specifically, we use the PAC-Bayes–λ bound from Theorem 6 in Thiemann et al. (2017) or, equivalently, Theorem 3.30 from Seldin (2025), which states that for any probability distribution π over $\mathcal{H}$ that is independent of S and any $\delta \in (0, 1)$, with probability greater than $1 - \delta$ over a random draw of a sample S, for all distributions ρ over $\mathcal{H}$ and $\lambda \in (0, 2)$ simultaneously, it then holds that $\mathbb{E}_\rho\big[L(h)\big]$ is bounded by

$$\mathbb{E}_\rho\big[L(h)\big] \;\le\; \frac{\mathbb{E}_\rho\big[\hat{L}^{\mathrm{val}}(h, S)\big]}{1 - \frac{\lambda}{2}} \;+\; \frac{\mathrm{KL}\big(\rho\|\pi\big) + \ln\!\Big(\frac{2\sqrt{n-r}}{\delta}\Big)}{\lambda\,\big(1 - \frac{\lambda}{2}\big)\,(n - r)}. \qquad (\star)$$

[1]The dataset is downloaded from `https://archive.ics.uci.edu/ml/datasets/Ionosphere`.

Since $\mathbb{E}_\rho\big[\hat{L}^{\text{val}}(h,S)\big]$ is linear in ρ, and $\text{KL}(\rho||\pi)$ is convex in ρ by Thiemann et al. (2017), for a fixed λ, the right hand side of $(\star)$ is convex in ρ, and the minimum is achieved by

$$\rho_\lambda(h) = \frac{\pi(h)e^{-\lambda(n-r)\hat{L}^{\text{val}}(h,S)}}{\mathbb{E}_\pi\big[e^{-\lambda(n-r)\hat{L}^{\text{val}}(h',S)}\big]},$$

where $\mathbb{E}_\pi[e^{-\lambda(n-r)\hat{L}^{\text{val}}(h',S)}]$ the normalization factor, which covers continuous and discrete hypothesis spaces in a unified notation. In our case, we have that

$$\mathbb{E}_\pi[e^{-\lambda(n-r)\hat{L}^{\text{val}}(h',S)}] = \sum_{h'\in\mathcal{H}} \pi(h')e^{-\lambda(n-r)\hat{L}^{\text{val}}(h',S)},$$

which means that the update rule for ρ_λ is given by

$$\rho_\lambda(h) = \frac{\pi(h)e^{-\lambda(n-r)\hat{L}^{\text{val}}(h,S)}}{\sum_{h'\in\mathcal{H}}\pi(h')e^{-\lambda(n-r)\hat{L}^{\text{val}}(h',S)}}.$$

However, as the hint in the exercise tells us, direct computation of ρ_λ is numerically unstable, since it leads to division of zero by zero for large $n-r$. To fix this, we normalize by $e^{-\lambda(n-r)\hat{L}^{\text{val}}_{\text{min}}}$, where $\hat{L}^{\text{val}}_{\text{min}} = \min_h \hat{L}^{\text{val}}(h,S)$, such that the expression for ρ_λ becomes

$$\rho_\lambda(h) = \frac{\pi(h)e^{-\lambda(n-r)\hat{L}^{\text{val}}(h,S)}}{\sum_{h'\in\mathcal{H}}\pi(h')e^{-\lambda(n-r)\hat{L}^{\text{val}}(h',S)}} = \frac{\pi(h)e^{-\lambda(n-r)\left(\hat{L}^{\text{val}}(h,S)-\hat{L}^{\text{val}}_{\text{min}}\right)}}{\sum_{h'\in\mathcal{H}}\pi(h')e^{-\lambda(n-r)\left(\hat{L}^{\text{val}}(h',S)-\hat{L}^{\text{val}}_{\text{min}}\right)}}$$

that does not lead to numerical instability problems. Next, we note that for $t\in(0,1)$ and $c_1,c_2\geq 0$ the function $\frac{c_1}{1-t}+\frac{c_2}{t(1-t)}$ is convex in t, such that that the right hand side of the inequality $(\star)$ is convex in λ for $\lambda\in(0,2)$ and fixed ρ, so that the minimum is achieved by

$$\lambda = \frac{2}{\sqrt{\frac{2(n-r)\mathbb{E}_\rho[\hat{L}^{\text{val}}(h,S)]}{\left(\text{KL}(\rho||\pi)+\ln\frac{2\sqrt{n-r}}{\delta}\right)}+1}+1}.$$

Thus, we minimize the bound in $(\star)$ by alternating application of the update rules for ρ_λ and λ given above. In the next subsections, we show how these are implemented in Python.

Construction of a Hypothesis Space

When performing the experiment from the paper, we will train m weak classifiers and weight their predictions through minimization of the PAC-Bayes-λ bound. However, computation of the normalizing constant (partition function) in PAC-Bayes requires summing over all hypotheses, which is intractable when $\mathcal{H}$ is infinite. Instead, we build a finite hypothesis space $\mathcal{H}$ by training m hypotheses, where each of them are trained on r random points from S and validated on the remaining $n-r$ points. For the `Ionosphere` dataset, we first have that $n=200$, and in the paper they then take $r=d+1=34+1=35$.

Formally, we first let $h \in \{1, \ldots, m\}$ index the hypotheses in $\mathcal{H}$. Then, we let S_h denote the training set of h and $S \setminus S_h$ the validation set, where $S_h \subset S$ with $|S_h| = r$. By letting the validation error of h be $(n - r)$ i.i.d. random variables with bias $L(h)$, we have that

$$\hat{L}^{\text{val}}(h, S) = \frac{1}{n - r} \sum_{(X,Y) \in S \setminus S_h} \ell(h(X), Y),$$

and since we focus on the zero-one loss, then $\ell(h(X), Y) = \mathbf{1}[h(x) \neq y]$, such that it has value 1 if the predicted label is incorrect and value 0 if the predicted label is correct.

Implementation of the Experiment
When we have to implement the PAC-Bayesian weighting of weak classifiers and replicate the experiment done in the paper that we introduced in the last subsection in Python, we proceed in two parts: first a single-classifier baseline using an RBF-kernel SVM tuned by cross-validation, and then the ensemble with PAC-Bayes–λ aggregation, where the baseline is used as a comparison of the prediction accuracy and runtime of our prediction strategy.

Baseline: Here, we use a radial basis function (RBF) kernel SVM with a 5-fold cross-validation (CV) procedure by solving (see question 2 from home assignment 3) the problem

$$\min_{w,b} \quad \frac{1}{2}\|w\|^2 + C \sum_{i=1}^{n} \max\left(0, 1 - y_i(w^\top \phi_\gamma(x_i) + b)\right),$$

where ϕ_γ is the RBF feature-map with kernel $k(x, x') = \exp(-\gamma\|x - x'\|^2)$. As in the paper, we choose these values of the soft-margin parameter C and kernel-bandwidth parameter γ

$$\log_{10} C \in \{-3, -2, \ldots, 3\}, \qquad \gamma \in \{\gamma_J \cdot 10^{-4}, \gamma_J \cdot 10^{-2}, \ldots, \gamma_J \cdot 10^{4}\},$$

via a 5-fold CV on the $n = 200$ training points, where the seed γ_J is given by

$$\gamma_J = \frac{1}{2 \cdot \text{median}(G)^2}, \quad G(x_i) = \min_{(x_i, y_j) \in S \wedge y_i \neq y_j} \|x_i - x_j\|, \quad i \in \{1, \ldots, n\}$$

which provides the anchor for the search grid in the cross-validation. We then retrain on all $n = 200$ points with the best (C, γ) pair, measure its test zero–one loss given by

$$L_{\text{CV-SVM}} = \frac{1}{|T|} \sum_{(x,y) \in T} \mathbf{1}\big[\hat{y} \neq y\big],$$

and then we record its training time t_{CV}. The implementation in Python is seen below:

```python
def median_gamma(X, y):
    n = X.shape[0]
    d2 = np.sum((X[:, None] - X[None, :]) ** 2, axis=2)
    mask = y[:, None] != y[None, :]
    mins = np.min(np.where(mask, d2, np.inf), axis=1)
    return 1.0 / (2 * np.median(mins))


def baseline_cv_svm(X_tr, y_tr, X_te, y_te, reps=10, seed=0):
    gamma0 = median_gamma(X_tr, y_tr)
    gamma_grid = gamma0 * np.logspace(-4, 4, 9)
    C_grid = np.logspace(-3, 3, 7)

    mean_losses = []
    mean_times = []

    for _ in range(reps):
        svc = SVC(kernel="rbf")
        param_grid = {"C": C_grid, "gamma": gamma_grid}
        grid = GridSearchCV(svc, param_grid, cv=5, n_jobs=-1)

        t0 = time.perf_counter()
        grid.fit(X_tr, y_tr)
        clf = grid.best_estimator_
        elapsed = time.perf_counter() - t0

        loss = np.mean(clf.predict(X_te) != y_te)
        mean_losses.append(loss)
        mean_times.append(elapsed)

    return (
        np.mean(mean_losses),
        np.std(mean_losses),
        np.mean(mean_times),
        np.std(mean_times),
        grid.best_params_["C"],
        grid.best_params_["gamma"],
    )
```

Listing 2: Implementation of RBF-kernel SVM with 5-fold cross validation.

Here, we used `scikit-learn`'s `SVC` (from `sklearn.svm`) with an RBF kernel to train our support vector machines, `scikit-learn`'s `GridSearchCV` (from `sklearn.model_selection`) to perform a parallelized 5-fold cross-validation over the C and γ hyperparameter grids (automatically selecting the best combination), and `time.perf_counter` from the `time` module to measure the elapsed time for the entire fit procedure in each repetition.

PAC-Bayesian Aggregation: Here, we first select without labels m i.i.d. subsets $S_1, \ldots, S_m \subset S$ of size r and train a radial basis function (RBF) kernel SVM for each subset S_i with the same kernel $k(x, x') = \exp(-\gamma \|x - x'\|^2)$ as for the baseline. Also, the kernel-bandwidth γ is randomly selected for each subset S_i from the same grid that was used in the baseline. We compute its validation error on $S \setminus S_i$ for $i \in \{1, \ldots, m\}$ by

$$\hat{L}^{\text{val}}(h_i, S_i) = \frac{1}{n - r} \sum_{(x,y) \in S \setminus S_i} \mathbf{1}\left[h_i(x) \neq y\right]$$

on the $n - r$ points that are not used for training, and collect the vector $(\hat{L}^{\text{val}}(h_i))_{i=1}^{m}$.

The weighting of classifiers ρ is then computed through alternating minimization of the bound in $(\star)$. Specifically, with a uniform prior $\pi(h) = 1/m$ and confidence $\delta = 0.05$, the PAC-Bayes-λ bound in $(\star)$ is minimized by alternating the updates of ρ_λ and λ given by

$$\rho_{\lambda,i}^{(t+1)} = \frac{\exp\left[-\lambda^{(t)}(n-r)\left(\hat{L}_{\text{val}}(h_i, S_i) - \hat{L}_{\min}\right)\right]}{\sum_{j=1}^{m} \exp\left[-\lambda^{(t)}(n-r)\left(\hat{L}_{\text{val}}(h_j, S_j) - \hat{L}_{\min}\right)\right]}$$

$$\lambda^{(t+1)} = \frac{2}{\sqrt{\frac{2(n-r)\mathbb{E}_\rho[\hat{L}^{\text{val}}(h_i,S_i)]}{\left(\text{KL}(\rho^{(t+1)}||\pi)+\ln\frac{2\sqrt{n-r}}{\delta}\right)}+1}+1},$$

where $\hat{L}_{\min} = \min_i \hat{L}^{\text{val}}(h_i, S_i)$. We initialize $\lambda^{(0)} = 1$ and stop when $|\lambda^{(t+1)} - \lambda^{(t)}| < 10^{-6}$. Upon convergence we obtain the optimal pair (ρ^*, λ^*) that minimizes the bound. Once (ρ^*, λ^*) is found, we form the ρ-weighted majority vote (see (3.31) in Seldin (2025)) by

$$\text{MV}_\rho(x) = \hat{y} = \text{sign}\left(\sum_{i=1}^{m} \rho_i^* h_i(x)\right)$$

and compute its test loss

$$L_{\text{PAC-B-Agg}} = \frac{1}{|T|} \sum_{(x,y)\in T} \mathbf{1}[\hat{y} \neq y],$$

as well as the total runtime t_m (training all m SVMs plus the bound-minimization). The implementation of the PAC-Bayesian Aggregation method in Python is seen below:

```
def pacbayes_aggregation(X_tr, y_tr, X_te, y_te, ms, r, delta, gamma_grid, C, reps, seed):
    n = len(y_tr)
    mean_losses, std_losses = [], []
    mean_times, std_times = [], []
    mean_bounds, std_bounds = [], []
    rng = np.random.RandomState(seed)

    for m in ms:
        losses, times, bounds = [], [], []
        for _ in range(reps):
            t0 = time.perf_counter()
            # Draw m subsets of r samples
            subsets = [rng.choice(n, r, replace=False) for _ in range(m)]
            # Compute validation losses and test predictions
            val_losses = np.zeros(m)
            test_preds = np.zeros((m, len(y_te)))
            for i, idx in enumerate(subsets):
                gamma = rng.choice(gamma_grid)
                clf = SVC(kernel="rbf", C=C, gamma=gamma)
                clf.fit(X_tr[idx], y_tr[idx])
                mask = np.ones(n, bool)
                mask[idx] = False
                val_losses[i] = np.mean(clf.predict(X_tr[mask]) != y_tr[mask])
                test_preds[i] = clf.predict(X_te)
```

```
            # Alternating minimization of PAC-Bayes-lambda
            lam = 1.0
            Lmin = val_losses.min()
            for _ in range(200):
                w = np.exp(-lam * (n - r) * (val_losses - Lmin))
                rho = w / w.sum()
                KL = np.sum(rho * np.log(rho * m))
                E_L = rho.dot(val_losses)
                lam_new = 2 / (np.sqrt(2 * (n - r) * E_L / (KL + np.log(2 * np.sqrt(n - r)
     / delta))+ 1)+ 1)
                if abs(lam_new - lam) < 1e-6:
                    lam = lam_new
                    break
                lam = lam_new
            # Compute rho-weighted vote loss and bound
            y_pred = np.sign(rho @ test_preds)
            losses.append(np.mean(y_pred != y_te))
            rhs = (KL + np.log(2 * np.sqrt(n - r) / delta)) / (n - r)
            bounds.append(invert_kl(rho.dot(val_losses), rhs))
            times.append(time.perf_counter() - t0)

        mean_losses.append(np.mean(losses))
        std_losses.append(np.std(losses))
        mean_times.append(np.mean(times))
        std_times.append(np.std(times))
        mean_bounds.append(np.mean(bounds))
        std_bounds.append(np.std(bounds))

    return (
        np.array(mean_losses), np.array(std_losses),
        np.array(mean_times), np.array(std_times),
        np.array(mean_bounds), np.array(std_bounds)
    )
```

Listing 3: Implementation of PAC-Bayesian Aggregation.

Here, we again used `scikit-learn`'s `SVC` with an RBF kernel to train each weak classifier on its random subset, and `time.perf_counter` to measure the total runtime for each trial, while the equations presented above have been implemented in the code to compute losses, do the alternating updates of ρ and λ, and aggregate the final loss, bound, and runtimes.

Performance of Experiment

To replicate Figure 2 of Thiemann et al. (2017) on the `Ionosphere` dataset, where the experiment focuses on the effect of increasing the number m of weak SVMs when their training sizes r are kept fixed, we have $n = 200$ and choose $r = d + 1 = 34 + 1 = 35$. Then, we run our training procedure over 20 logarithmically spaced integer values of $m \in [1, 200]$. For each m we train m weak SVMs on random r-subsets and then perform the PAC-Bayes–λ alternating minimization to update ρ_λ and λ. Here, we record the following quantities:

- $L_{\text{CV-SVM}} = \dfrac{1}{|T|} \displaystyle\sum_{(x,y)\in T} \mathbf{1}\big[\hat{y} \neq y\big]$.
- $L_{\text{PAC-B-Agg}} = \dfrac{1}{|T|} \displaystyle\sum_{(x,y)\in T} \mathbf{1}\Big[\operatorname{sign}\big(\sum_{i=1}^{m} \rho_i^* \, h_i(x)\big) \neq y\Big]$.

- t_m and t_{CV}, which are the total running times (in seconds) for the PAC-Bayes aggregation and CV-SVM training, respectively. Here, the running time for the baseline includes both cross-validation and training the final SVM on the entire training set. In contrast, the running time for PAC-Bayesian aggregation accounts for training the m weak SVMs, their validation, and finally the computation of ρ.

In addition to these quantities, following Thiemann et al. (2017), we also report the value of the PAC-Bayes-kl bound that by Theorem 2 in Thiemann et al. (2017) is given by

$$\mathrm{kl}\left(\mathbb{E}_\rho\left[\hat{L}^{\mathrm{val}}(h,S)\right]\middle\|\mathbb{E}_\rho[L(h)]\right) \leq \frac{\mathrm{KL}(\rho||\pi) + \ln\frac{2\sqrt{n-r}}{\delta}}{n-r}$$

with probability at least $1-\delta$ for all ρ simultaneously. This yields the tightest PAC-Bayes–kl bound on the true randomized loss, where we numerically solve for

$$q_{\mathrm{kl}} = \min\left\{q \in [p,1]\colon\ \mathrm{kl}\left(\mathbb{E}_\rho[\hat{L}^{\mathrm{val}}(h,S)]\middle\|q\right) \leq \frac{\mathrm{KL}(\rho\|\pi) + \ln\frac{2\sqrt{n-r}}{\delta}}{n-r}\right\},$$

where $\mathrm{kl}(p\|q) = p\ln\frac{p}{q} + (1-p)\ln\frac{1-p}{1-q}$. Although Theorem 3.31 in Seldin (2025) shows that the zero–one loss of the ρ–weighted majority vote is at most $2\,q_{\mathrm{kl}}$, in accordance with Thiemann et al. (2017), we report q_{kl} itself as the "PAC-Bayes-kl" curve in Figure 2. In the implementation, we use a numerical procedure that performs bisection to find the value of q_{kl} satisfying the PAC-Bayes-kl bound by iteratively solving for the root of a KL divergence expression, where the divergence between the two Bernoulli parameters is computed and compared against a given threshold until convergence, see `invert_kl(p, rhs)` and `kl_div` in `exercise_2.py`. The full implementation of the second experiment in Thiemann et al. (2017) and the replication of Figure 2 with the five above quantities in Python is seen below:

```
X_tr, X_te, y_tr, y_te = load_ionosphere()
n, d = X_tr.shape
r = d + 1
delta = 0.05

# Baseline
L_cv, Lcv_std, t_cv, tcv_std, best_C, best_gamma = baseline_cv_svm(
    X_tr, y_tr, X_te, y_te, reps=10, seed=42)

# PAC-Bayesian Aggregation
ms = np.unique(np.round(np.logspace(0, np.log10(n), 20))).astype(int)
gamma0 = median_gamma(X_tr, y_tr)
gamma_grid = gamma0 * np.logspace(-4, 4, 9)
L_m, L_std, t_m, t_std, q_m, q_std = pacbayes_aggregation(
    X_tr, y_tr, X_te, y_te, ms, r, delta, gamma_grid, best_C, reps=10, seed=42)

plot_no_std(ms, L_cv, L_m, q_m, t_cv, t_m)
plot_with_std(ms, L_cv, L_m, L_std, q_m, q_std, t_cv, t_m, t_std)
```

Listing 4: Implementation of the paper's second experiment and replication of its Figure 2.

Here, we perform the experiment by first calling loading and preprocesssing the data and generate the 20 values of $m \in [1, 200]$. Then, we get the mean $\pm$ std of the CV-tuned SVM's loss and runtime for 10 repetitions of the experiment, and the PAC-Bayesian aggregation ensemble's loss, bound, and runtime. Finally, we plot everything via `plot_no_std` (which shows the mean test-loss curves of the CV-SVM, the PAC-Bayes aggregation, and the PAC-Bayes-kl bound, with their runtimes) and `plot_with_std` (which overlays the same curves but adds $\pm$ standard-deviation error bars for our PAC-Bayes aggregation's test loss and runtime, plus a shaded band for the bound's variability) functions (see `exercise_2.py`).

In the following two plots, solid lines show the test loss, where we have red for CV-SVM, black for the ρ-weighted majority vote "Our Method" (minimizing the PAC-Bayes-λ bound), and blue for the PAC-Bayes-kl bound, while dashed lines show runtime (red for CV-SVM, black for PAC-Bayesian aggregation), as functions of the hypothesis set size m:

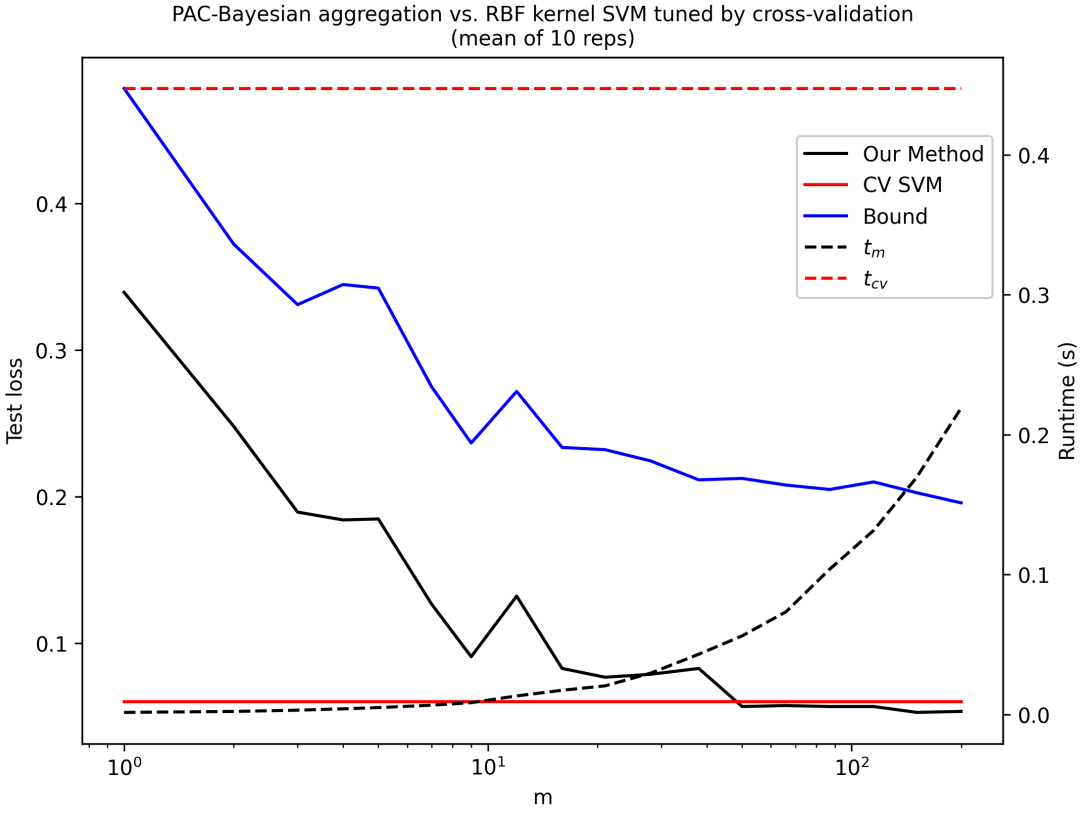

Figure 1: Comparison of PAC-Bayesian aggregation with RBF kernel SVM tuned by cross-validation (mean of 10 repetitions).

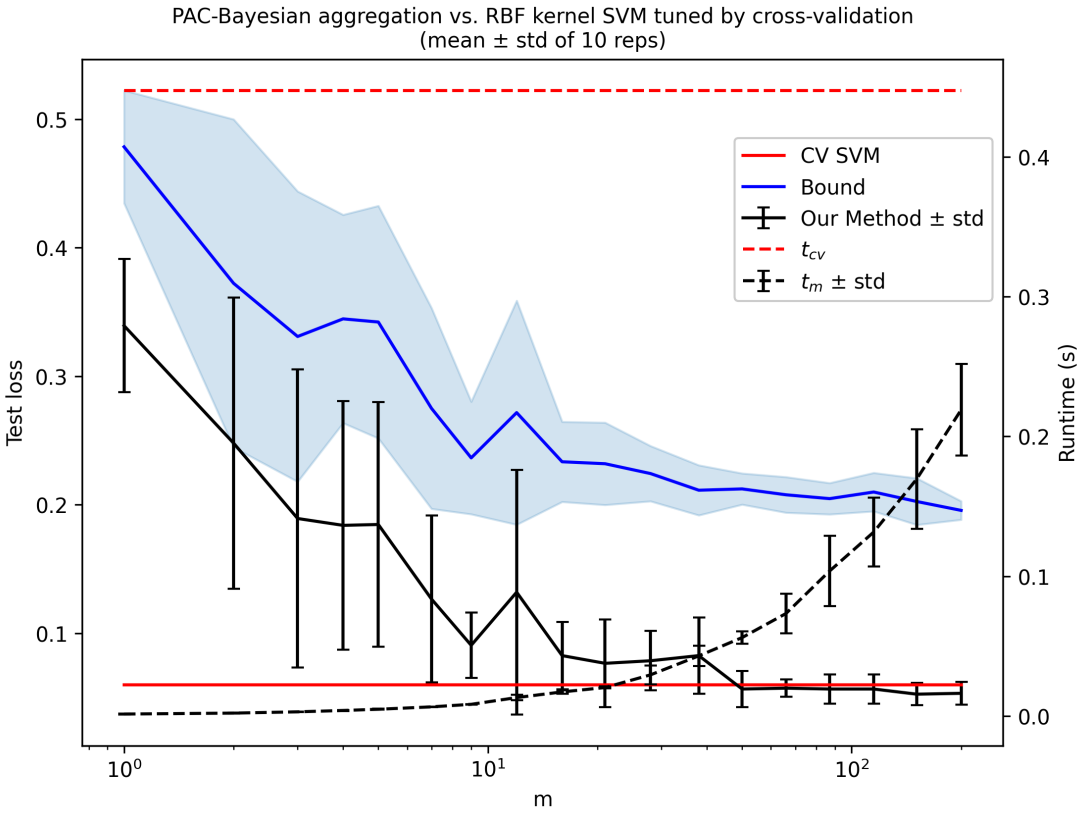

Figure 2: Comparison of PAC-Bayesian aggregation with RBF kernel SVM tuned by cross-validation (mean ± standard deviation of 10 repetitions).

Figure 1 and Figure 2 show that we have successfully reproduced the original experiment and replicated Thiemann et al. (2017)'s Figure 2 on the `Ionosphere` dataset. In particular, we see that even with relatively small values of m, the ρ-weighted vote (black curve) nearly matches the cross-validated SVM baseline (red line) in test accuracy, while incurring lower runtime (dashed black vs. dashed red). Our measured runtime for the CV-SVM is somewhat lower than reported in the paper, likely due to using a more optimized `scikit-learn` implementation and parallel grid search on newer hardware by passing `n_jobs=-1` into `GridSearchCV`. Moreover, the PAC-Bayes-kl bound (blue curve and shaded band) tracks the empirical loss very closely, demonstrating that the bound is very tight in this setting.

2 VC Dimension

Response to Question 3.1. We let $\mathcal{H}$ be a finite hypothesis set with $|\mathcal{H}| = M$ distinct hypotheses. We then recall that for any set of inputs $S = \{x_1, \ldots, x_n\}$, the number of distinct labelings (dichotomies) that $\mathcal{H}$ can realize on S is at most given by

$$m_{\mathcal{H}}(n) = |\{(h(x_1), \ldots, h(x_n)) \colon h \in \mathcal{H}\}| \leq |\mathcal{H}| = M,$$

since each hypothesis can contribute at most one labeling. By Definition 3.11 in Seldin (2025), $\mathcal{H}$ shatters S only if it realizes all 2^n possible binary labels on n points, such that

$$m_{\mathcal{H}}(n) = 2^n.$$

But since $m_{\mathcal{H}}(n) \leq M$, shattering n points forces that

$$2^n \leq M \Longrightarrow n \leq \log_2 M.$$

Therefore no set of size $n > \log_2 M$ can be shattered by $\mathcal{H}$. Now, it follows that the VC-dimension $d_{\mathrm{VC}}(\mathcal{H})$ of $\mathcal{H}$, which by Definition 3.12 in Seldin (2025) is given by

$$d_{\mathrm{VC}}(\mathcal{H}) = \max\{n \colon m_{\mathcal{H}}(n) = 2^n\},$$

then satisfies the lower bound

$$d_{\mathrm{VC}}(\mathcal{H}) \leq \log_2 M.$$

Response to Question 3.2. We let $\mathcal{H}$ be a hypothesis space with 2 hypotheses (i.e., $|\mathcal{H}| = 2$). Now, we have to prove that $d_{\mathrm{VC}}(\mathcal{H}) = 1$.

Proof. From the response to Question 3.1, we know that the lower bound of the VC dimension with $|\mathcal{H}| = 2$ distinct hypotheses is given by

$$d_{\mathrm{VC}}(\mathcal{H}) \leq \log_2 |\mathcal{H}| = \log_2 2 = 1.$$

Now, let $\{h_1, h_2\}$ with $h_1 \neq h_2$. Since the two hypotheses in $\mathcal{H}$ are distinct by definition, there is at least one point $x^* \in \mathcal{X}$ on which they differ, such that $h_1(x^*) \neq h_2(x^*)$.

Pick that point x^* and consider the singleton $S = \{x^*\}$. There are exactly $2^1 = 2$ possible labelings of S, namely $\{x^*\} \mapsto 0$ and $\{x^*\} \mapsto 1$. But on x^*, the two hypotheses realize both of these labelings, such that $(h_1(x^*))$ and $(h_2(x^*))$ are different, so $\mathcal{H}$ indeed produces both possible binary labels on S. Hence, $\mathcal{H}$ shatters the singleton S, and so

$$d_{\mathrm{VC}}(\mathcal{H}) \geq 1.$$

By combining the two bounds, we can then conclude that $d_{\mathrm{VC}}(\mathcal{H}) = 1$ as claimed. □

Response to Question 3.3. We let $\mathcal{H}_+$ be the class of "positive" circles in $\mathbb{R}^2$, where each $h \in \mathcal{H}_+$ is defined by the center of the circle $c \in \mathbb{R}^2$ and its radius $r \in \mathbb{R}$. All points inside the circle are labeled positively and outside negatively.

Now, we have to prove that the VC dimension $d_{\mathrm{VC}}(\mathcal{H}_+) \geq 3$.

Proof. To show that $d_{\text{VC}}(\mathcal{H}_+) \geq 3$, we will construct a specific set of three non-collinear points in $\mathbb{R}^2$, which can be shattered by the hypothesis class $\mathcal{H}_+$ of "positive" circles (i.e., those that label points inside the circle as $+1$ and points outside as -1). Thus, we choose

$$P_0 = (0, 0), \quad P_1 = (1, 0), \quad P_2 = (0, 1)$$

which form a right triangle. We know that there are $m_{\mathcal{H}_+}(3) = 2^3 = 8$ possible assignments of the labels $\{-1, +1\}$ to the three points. We now list how to find, in each case, a circle in $\mathcal{H}_+$ that includes exactly those points labelled $+1$ below:

- **No points labeled +1**: A tiny circle placed far from $\{P_0, P_1, P_2\}$ makes all three lie outside (label -1).
- **Exactly one point labeled +1**: For each $i \in \{0, 1, 2\}$, a sufficiently small circle around P_i picks out that single positive.
- **Exactly two points labeled +1**: For each pair $\{P_i, P_j\}$, one can draw a circle that contains exactly those two and excludes the third.
- **All three points labeled +1**: The circumcircle of the triangle $P_0P_1P_2$ contains all three vertices.

All eight of these cases are illustrated in Figure 3, with each subplot showing the unique "positive" circle whose interior is exactly the set of points labeled $+1$. In the figure, points labeled $+1$ are shown in blue and points labeled -1 in red. The figure has been generated in Python, and the code can be found in `exercise_3.py`. Thus, the set $\{P_0, P_1, P_2\}$ is shattered by $\mathcal{H}_+$ by definition, and so we have that $d_{\text{VC}}(\mathcal{H}) \geq 3$ as claimed. □

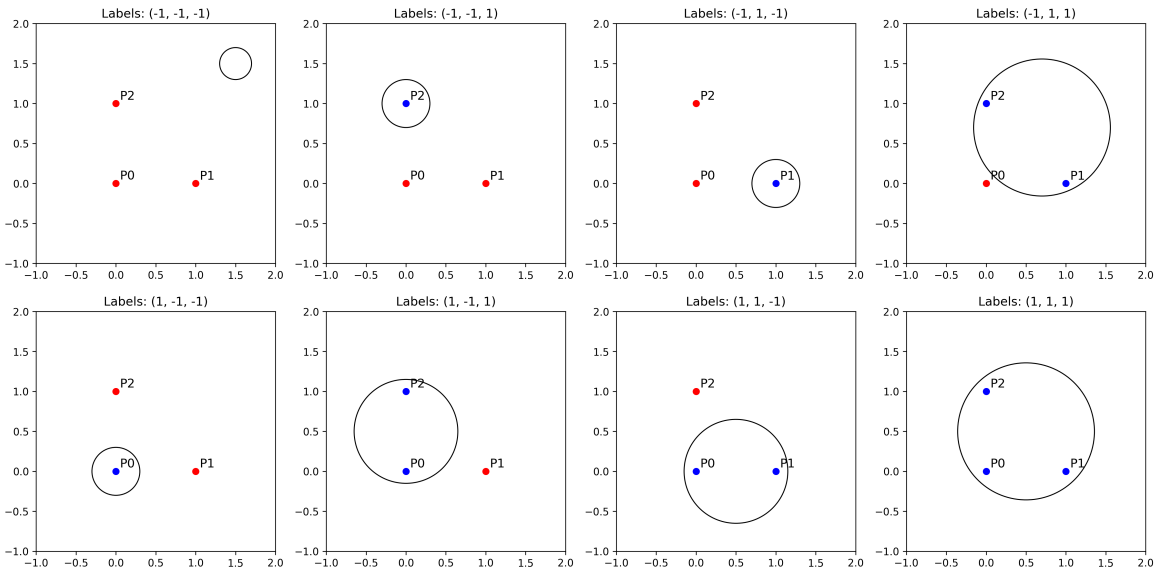

Figure 3: All $2^3 = 8$ labelings of $\{P_0, P_1, P_2\}$ realized by circles in $\mathcal{H}_+$.

Response to Question 3.4. We let $\mathcal{H} = \mathcal{H}_+ \cup \mathcal{H}_-$ be the class of "positive" and "negative" circles in $\mathbb{R}^2$, where the "negative" circles are negative inside and positive outside.

Now, we have to prove that the VC dimension $d_{\text{VC}}(\mathcal{H}) \geq 4$.

Proof. We let $\mathcal{H} = \mathcal{H}_+ \cup \mathcal{H}_-$ be the class of "positive" and "negative" circles in $\mathbb{R}^2$, where

- each $h \in \mathcal{H}_+$ labels points inside the circle as $+1$ and outside as -1,
- each $h \in \mathcal{H}_-$ labels points inside the circle as -1 and outside as $+1$.

We will now show that the VC dimension is at least four, i.e., $d_{\text{VC}}(\mathcal{H}) \geq 4$. We take

$$P_0 = (0,0), \quad P_1 = (0,1), \quad P_2 = (-1,2), \quad P_3 = (1,2).$$

No three of these lie on a common line. There are $m_{\mathcal{H}}(4) = 2^4 = 16$ possible assignments of the labels to the four $\{P_0, P_1, P_2, P_3\}$. Given any assignment, we let

$$S = \{P_i \colon \text{ label of } P_i = +1\}, \quad S^c = \{P_i \colon \text{ label of } P_i = -1\}.$$

If S can be enclosed in a single circle, then we use that circle as a $\mathcal{H}_+$-hypothesis. In our illustrations, this circle is drawn solid black, coloring the $+1$ points blue and the -1 points red. Otherwise, the complement S^c must be enclosable in a single circle. Then, we take that circle as a $\mathcal{H}_-$-hypothesis, which realizes the labeling by making its interior the -1 points and its exterior the $+1$ points. In our illustrations, this circle is drawn solid red.

All these sixteen configurations appear in Figure 4. In each subplot the four points are colored blue ($+1$) or red (-1), and the single solid circle, which is black for a positive-circle hypothesis, and red for a negative-circle hypothesis, realizes exactly that labeling. Because we have found, for every one of the 16 labelings, a hypothesis in $\mathcal{H}$ that matches it, the set $\{P_0, P_1, P_2, P_3\}$ is shattered by $\mathcal{H}$. Thus, we have $d_{\text{VC}}(\mathcal{H}) \geq 4$ by definition as claimed. □

Response to Question 3.5. We know that if a hypothesis space $\mathcal{H}$ has an infinite VC-dimension, it is possible to construct a worst-case data distribution that will lead to overfitting, i.e., with probability at least $\frac{1}{8}$ it will be possible to find a hypothesis for which $L(h) \geq \hat{L}(h, S) + \frac{1}{8}$. Now, we have to construct a data distribution $p(X, Y)$ and a hypothesis space $\mathcal{H}$ with infinite VC-dimension, such that for any sample S of more than 100 points with probability at least 0.95, we will have $L(h) \leq \hat{L}(h, S) + 0.01$ for all $h \in \mathcal{H}$.

First, we let the instance space be $\mathcal{X} = \mathbb{R}$, and let the joint distribution p concentrate all its mass on a single input point $x_0 \in \mathbb{R}$. Concretely, we then take

$$P_X(\{x_0\}) = 1, \quad P(Y = 0 | X = x_0) = 1.$$

Thus, every draw from p yields the same example $(x_0, 0)$. Next, we define the hypothesis class $\mathcal{H}$ to be the set of all functions $h \colon \mathbb{R} \to \{0, 1\}$. Since $\mathbb{R}$ is infinite, we can always

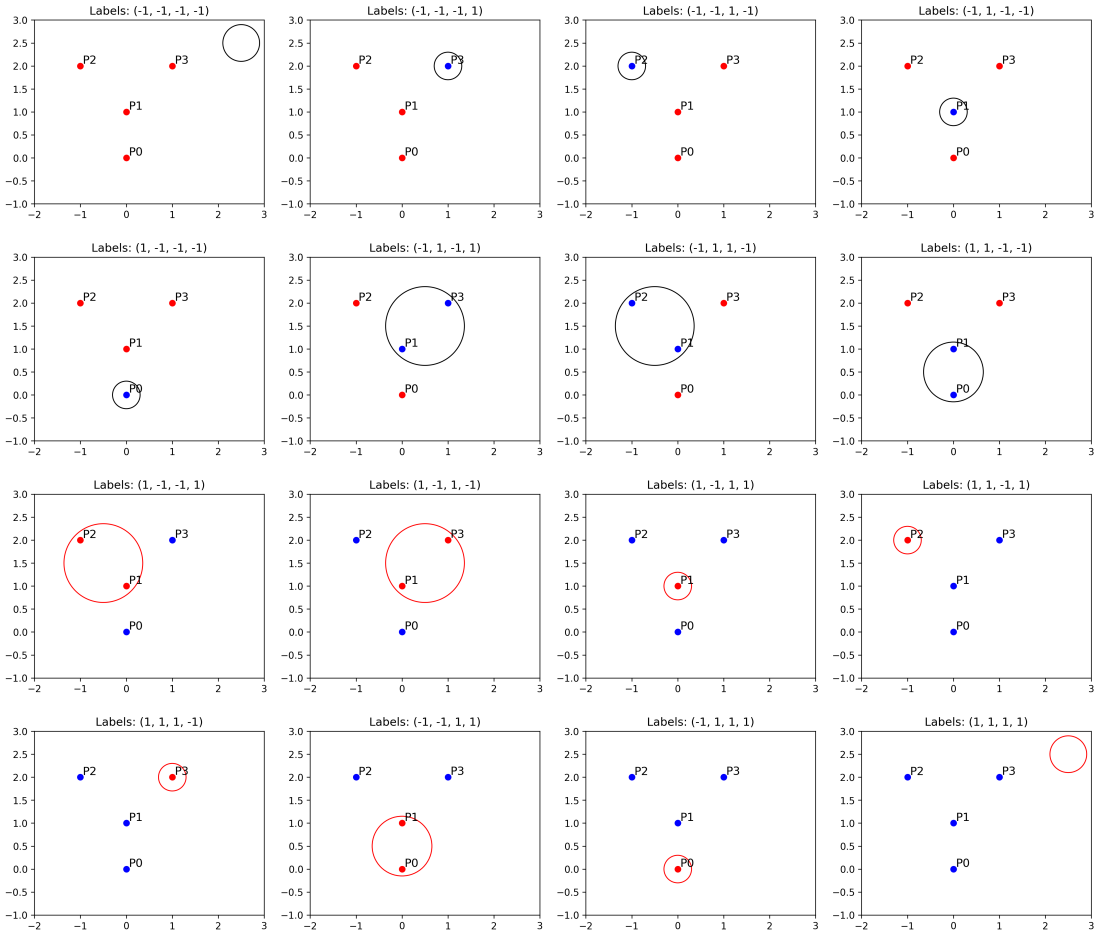

Figure 4: All $2^4 = 16$ labels of $\{P_0, P_1, P_2, P_3\}$ realized by hypotheses in $\mathcal{H} = \mathcal{H}_+ \cup \mathcal{H}_-$. Solid black circles are "positive" with $+1$ inside, and solid red circles are "negative" with -1 inside.

pick, for any finite set of inputs, a hypothesis in $\mathcal{H}$ that shatters those points. Hence, $d_{\mathrm{VC}}(\mathcal{H}) = \infty$. Under this degenerate distribution, however, every hypothesis in $\mathcal{H}$ behaves identically on the support of p. Indeed, for any $h \in \mathcal{H}$, we have that

$$L(h) = \Pr_{(X,Y)\sim p}[h(X) \neq Y] = \Pr[h(x_0) \neq 0] = \mathbf{1}[h(x_0) \neq 0],$$

and for any sample $S = ((x_1, y_1), \ldots, (x_n, y_n))$ of size n, we also have that

$$\hat{L}(h, S) = \frac{1}{n}\sum_{i=1}^{n} \mathbf{1}[h(x_i) \neq y_i] = \frac{1}{n}\sum_{i=1}^{n} \mathbf{1}[h(x_0) \neq y_i] = \mathbf{1}[h(x_0) \neq 0].$$

Hence for every $h \in \mathcal{H}$ and every sample S, the true error and empirical error coincide, such that $L(h) = \hat{L}(h, S)$. It follows immediately that for any $n > 100$, with probability 1 (and thus certainly ≥ 0.95) over the draw of S that we have

$$\sup_{h\in\mathcal{H}}\big[L(h) - \hat{L}(h, S)\big] \;=\; 0 \;\leq\; 0.01.$$

This example does not contradict the usual worst-case result for infinite VC-dimension, because that result presumes a richer distribution whose support can have arbitrarily complex labelings. By collapsing the data to a single labeled point, we force every $h \in \mathcal{H}$ to be indistinguishable on the observed sample, ensuring perfect convergence despite $d_{\mathrm{VC}}(\mathcal{H}) = \infty$.

References

Seldin, Y. (2025). Machine learning: The science of selection under uncertainty.

Thiemann, N., Igel, C., Wintenberger, O., and Seldin, Y. (2017). A strongly quasiconvex pac-bayesian bound.